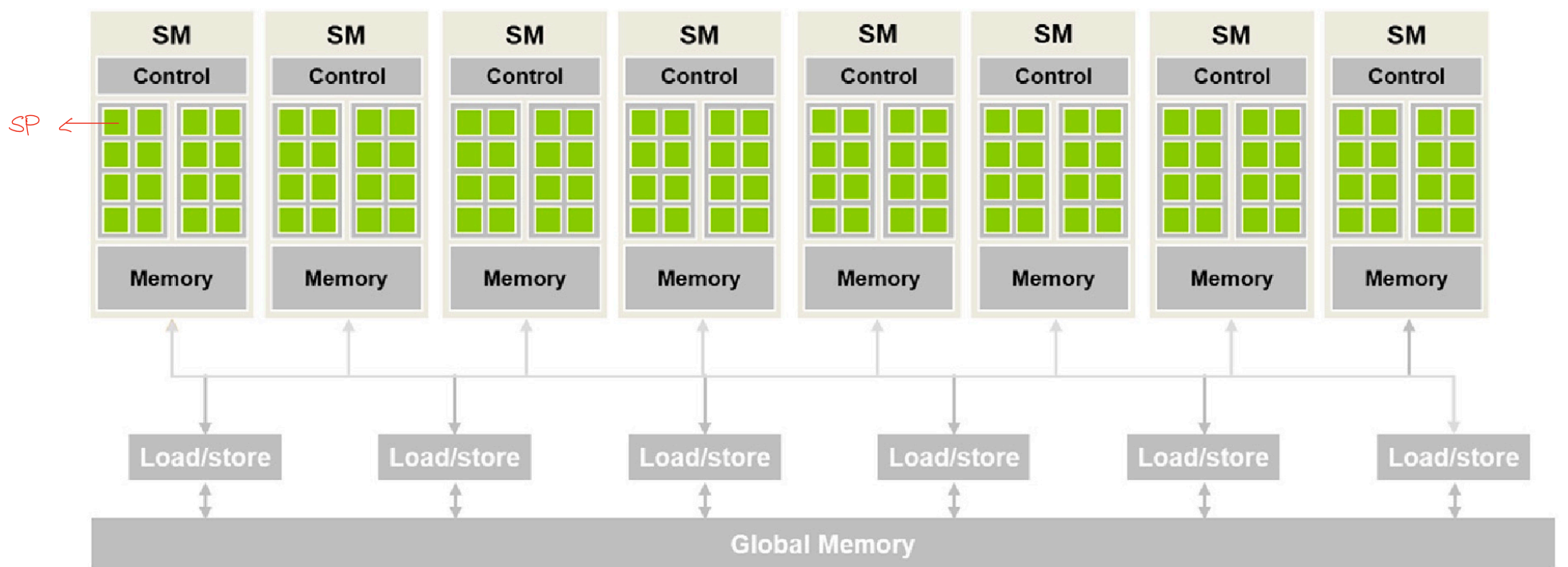


Architecture of modern GPUs



- A CUDA-enabled GPU is organized into an array of streaming multiprocessors (SMs)
- Each SM consists of several streaming processors or CUDA cores, that share some control & memory resources
- Further, each SM has on-chip memory, which is managed by the programmer instead of hardware in case of cache.

In generic terms, such an on-chip memory is called scratchpad memory.

Block Scheduling

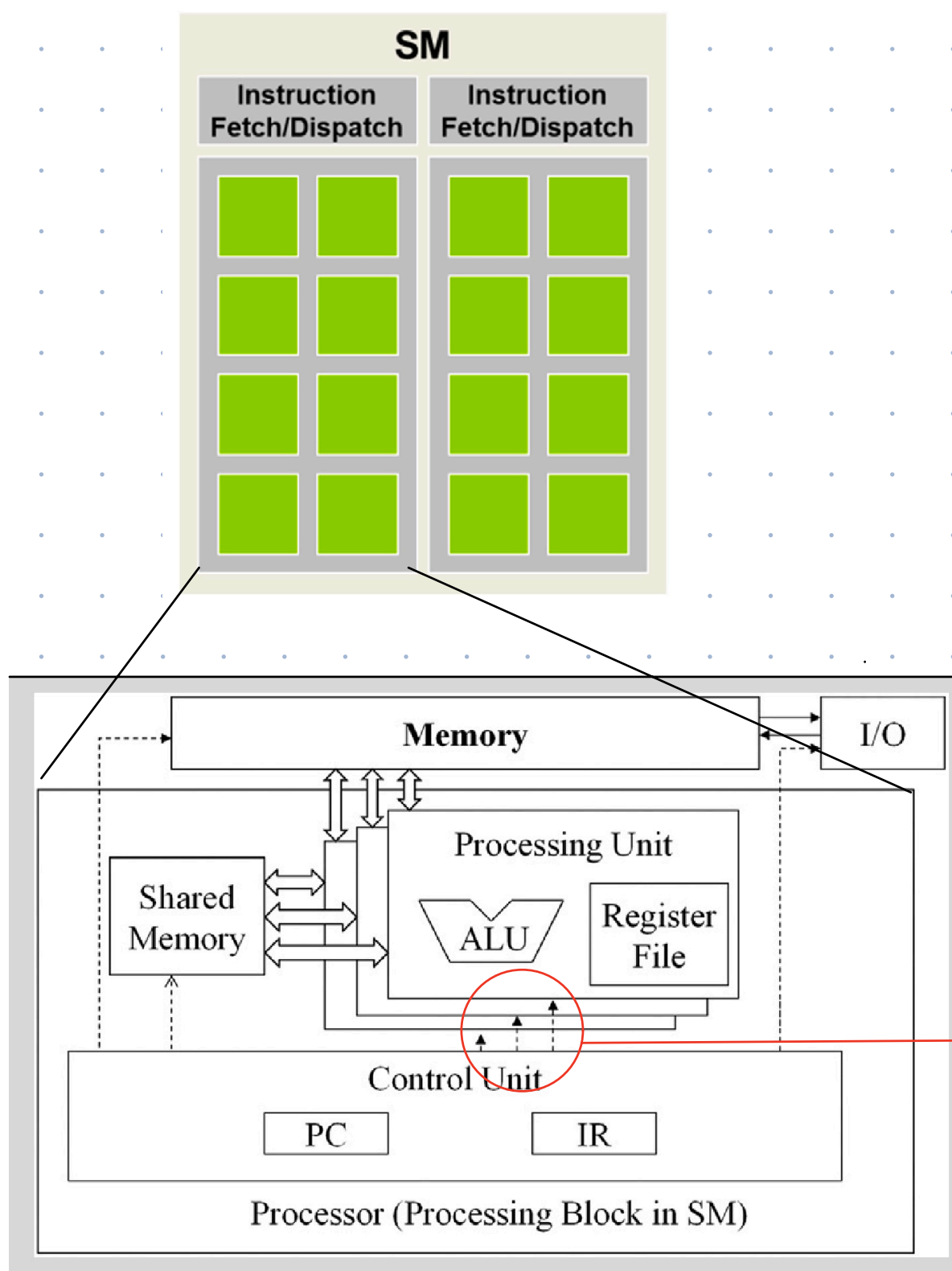
- Each block is assigned to one & only one SM, whereas a single SM may be assigned multiple blocks.
- There is a limit to number of blocks that can be assigned to an SM at a time.
- Due to this limit, the runtime maintains a list of blocks that need to execute & assign new blocks to SMs when previously assigned blocks complete execution.

Warpes And SIMD hardware

- Each block is divided into 32 thread groups called warps.

If there are insufficient threads to form a warp then inactive threads are added to get the full 32 thread warp.

- Cores in each SM are grouped to form processing blocks, and each share a Instruction fetch/Dispatch unit.
- All threads in same warp are assigned to the same processing block.
- The Fetch/Dispatch unit fetches the same instruction for all threads in the warp.



CU is shared among cores / Processing units

Control divergence

- SIMD works well when all threads within a warp follow the same execution path / control flow i.e either all threads in a warp execute if-path or else-path.
- However, when threads within a warp take different control flow paths, the SIMD hardware will take multiple passes through these paths, one pass for each path.

For example, for an if-else construct, one pass executes the threads that follow if-path, and other executes threads that follow the else-path.

During each pass, the threads that follow the other are not allowed to take effect.

- From Volta architecture onwards, the passes may be executed concurrently, meaning execution of one pass may be interleaved with the execution of another pass.

```
if(threadIdx.x < 24) {
```

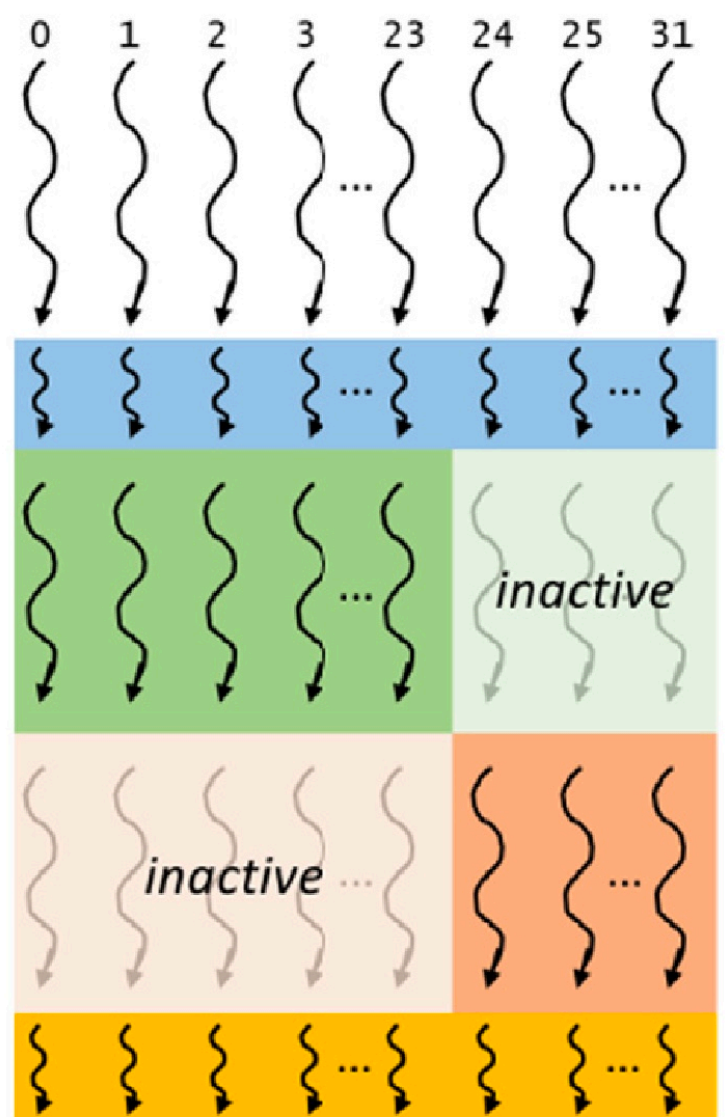
A

```
} else {
```

B

```
}
```

C



- If a control construct depends on threadIdx, then it can potentially cause control divergence.

Warp scheduling and latency tolerance

- When blocks are assigned to SMs, there are usually more threads than cores in the SM.

As such, each SM can execute only a subset of threads.

So, Why do we need so many warps assigned to an SM if it can execute only a subset of them at any instant?

- To tolerate long latency operations such as global memory accesses.
- When an instruction to be executed by a warp needs to wait for the result of a long-latency ops, then that warp is not selected.

Instead, another resident warp is scheduled based on a priority mechanism.

1. Global Scheduling: Blocks to SMs (The GigaThread Engine)

You are spot on here. At the very top level of the GPU, there is a master scheduler (NVIDIA calls this the GigaThread Engine).

- **What it does:** It looks at the massive grid of blocks in your kernel and assigns entire Thread Blocks to available SMs.
- **Dynamic:** As soon as an SM finishes a block and frees up memory/registers, the GigaThread Engine dynamically schedules another block onto that SM.

2. Allocation/Distribution: Blocks to Warps to PBs (Not quite a "Scheduler")

This is where the slight correction comes in. Once a block is handed to an SM, the process of turning it into warps and handing them to Processing Blocks (PBs) is an **allocation** step, not an active, cycle-by-cycle scheduling step.

- **Warp Creation:** The hardware automatically and strictly slices the block into warps of 32 threads based on their Thread IDs (Threads 0-31 are Warp 0, 32-63 are Warp 1, etc.). No decision-making happens here.
- **PB Assignment:** The SM's dispatch logic then distributes these warps across its PBs

- **Why it's not a scheduler:** Once a warp is assigned to a specific PB, it stays there until it finishes. It never jumps to another PB. Because there are no ongoing decisions being made over time, it's considered an initial distribution/allocation rather than a "scheduler."

3. Local Scheduling: The Warp Scheduler in the PB

You are spot on again. This is the cycle-by-cycle, highly active scheduler we talked about earlier.

- **What it does:** Inside the PB, the local warp scheduler looks at its assigned pool of resident warps every single clock cycle, checks which ones are ready to fire, and picks one to execute its next instruction on the PB's cores.

Resource partitioning and occupancy

- The execution resources in SM include registers, shared memory, thread block slots, and thread slots
- A100 GPU can support a maximum of:
 - 32 blocks per SM — thread block slots
 - 64 warps (2048 threads) per SM — thread slots
 - 1024 threads per block
 - 65,536 registers per SM

Case 1:

- Consider a grid launched with 1024 threads blocks, then the 2048 thread slot in each SM are partitioned & assigned to 2 blocks.

Similarly, in case of a block size of 512, 256, 128, or 64 threads, the 2048 thread slots are partitioned & assigned to 4, 8, 16, or 32 blocks respectively.

- In this case, the total number of threads assigned to the SM is 2048, so it maximizes occupancy.

Case 2:

- Consider a grid launched with 32 thread blocks, then 2048 thread slots will be divided among 64 blocks.

However, A100 supports a maximum of 32 blocks per SM.

This means only 1024 thread slots will be assigned.
Which gives an occupancy of $1024/2048 = 0.5$

To fully utilize the thread slots and achieve maximum occupancy, one needs atleast 64 threads in each block.

$$\begin{array}{l} \hookrightarrow 2048 \text{ thread slots} \\ \hline 32 \text{ thread block slots} \end{array}$$

Case 3:

- Consider a grid launched with 768 thread blocks. In this case, 2048 is not divisible by 768.
So,

$$\frac{2048}{768} = 2.67$$

So, each SM is assigned 2 blocks, which results in occupying 1536 thread slots out of 2048.

$$\frac{1536}{2048} = 0.75$$

Case 4:

Consider a kernel that requires 64 registers per thread. Thus, we can have $65536/64 = 1024$ threads per SM.

In this case, only 1024 out of 2048 thread slots are utilized.

Case 5:

Consider a kernel that requires 31 registers per thread, with block size 512.

So, each SM will have

$$\frac{2048}{512} = 4 \text{ blocks}$$

These threads will use a maximum of $2048 \times 31 = 63488$ registers, which is less than the limit.

Now, assume the code adds two more registers, which sends no. of registers per thread over the limit i.e. $2048 \times 33 = 67584$.

To solve this, the code runtime will schedule 3 blocks instead of 4 but now 1563 (3×512) out of 2048 will be used.

Exercise

```
1. 01  __global__ void foo_kernel(int* a, int* b) {
    02      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    03      if(threadIdx.x < 40 || threadIdx.x >= 104) {
    04          b[i] = a[i] + 1;
    05      }
    06      if(i%2 == 0) {
    07          a[i] = b[i]*2;
    08      }
    09      for(unsigned int j = 0; j < 5 - (i%3); ++j) {
    10          b[i] += j;
    11      }
    12  }
    13  void foo(int* a_d, int* b_d) {
    14      unsigned int N = 1024;
    15      foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d);
    16  }
```

a. threads per block = 128

So,

there are $\frac{128}{32} = 4$ warps per block

b. No. of blocks = $\frac{1024 + 128 - 1}{128}$

= 8

So, There are $8 \times 4 = 32$ warps in the block

c.

(i) 32

(ii) None.

(iii)

